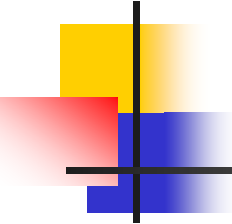


算法设计与分析

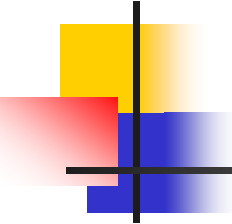
Analysis and Design of Algorithm

Lesson 12



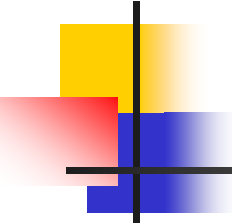
第四章小结

- 贪心法的适用性
 - **贪心选择性质**：短视地在每一步做出当前看来最优的选择，而不考虑这些选择会如何影响未来的选择。如果一个问题的全局最优解可以通过一系列局部最优解来构建且**无后效性**，那么该问题适用于贪心法。
 - **最优子结构**：如果一个问题的最优解包含了其子问题的最优解。这意味着可以通过组合子问题的最优解来构造原问题的最优解。
- 如果不满足上述两个性质，那么贪心法可能只能提供一个近似解或启发式解(多机调度问题)。



贪心法小结(cont.)

- 贪心法**正确性证明**
 - 数学归纳法（对算法步骤或者问题规模归纳）
 - 归纳基础、归纳假设、归纳验证
- 贪心法的优势：
 - 算法简单直观、时空复杂度低
- 几个著名的贪心算法
 - Huffman编码
 - 最小生成树的Prim算法和Kruskal算法
 - 单源最短路径的Dijkstra算法



课程内容

NP完全性理论与近似算法

算法高级理论

随机化算法

线性规划与网络流

高级算法

递归
分治

动态
规划

贪心
算法

回溯与
分支限界

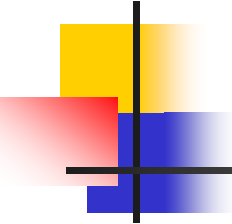
基础算法

算法分析与问题的计算复杂性

算法基础理论

第五章 回溯法

Backtracking



学习要点

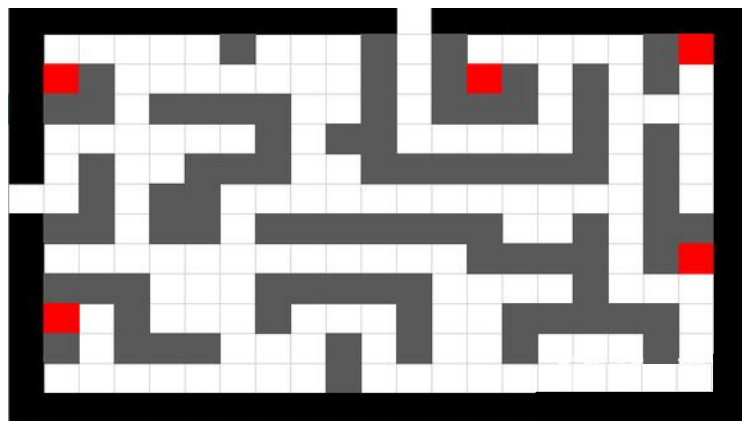
- **理解**回溯法的深度优先搜索策略
- **掌握**用回溯法解题的算法框架
 - **递归**回溯最优子结构性质
 - **迭代**回溯贪心选择性质
 - 子集树算法框架
 - 排列树算法框架
- 通过应用**范例学习**回溯法的设计策略
 - n 皇后问题、0-1背包问题、旅行售货员问题
 - 装载问题
 - 图着色问题

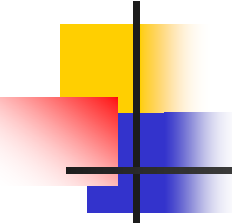
回溯法概述

回溯法

- 有许多问题，当需要找出它的解集或者要求回答什么解是满足某些约束条件的最佳解时，往往要使用回溯法。
- 回溯法的基本做法是**搜索**，一种组织得井井有条的（**带有系统性**），能避免不必要搜索（**带有跳跃性**）的穷举式搜索法。这种方法适用于解一些组合数相当大的问题。

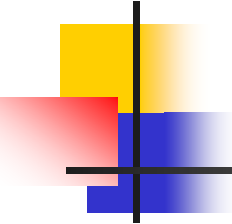
走迷宫游戏





回顾：最优化问题(第3章)

- 有 n 个输入（**解空间**），问题的解由这 n 个输入的一个子集组成，这个子集必须满足某些事先给定的条件，这些条件称为**约束条件**，满足约束条件的解称为问题的**可行解**。
- 满足约束条件的可行解可能不只一个，为了衡量这些可行解的优劣，事先给出一定的标准，这些标准通常以函数的形式给出，称为**目标函数**，使目标函数取得极值（极大或极小）的可行解称为**最优解**。
- 这类问题就称为**最优化问题**。

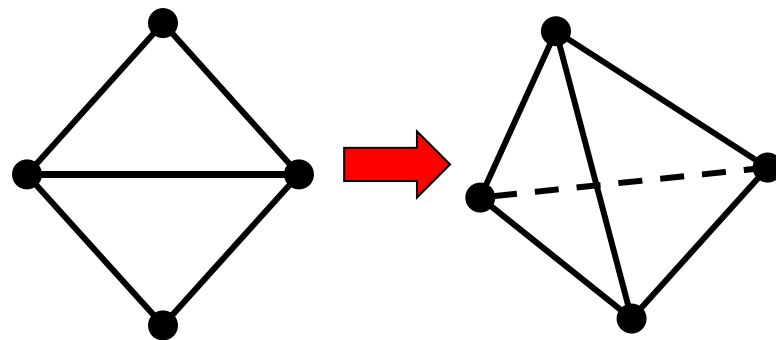
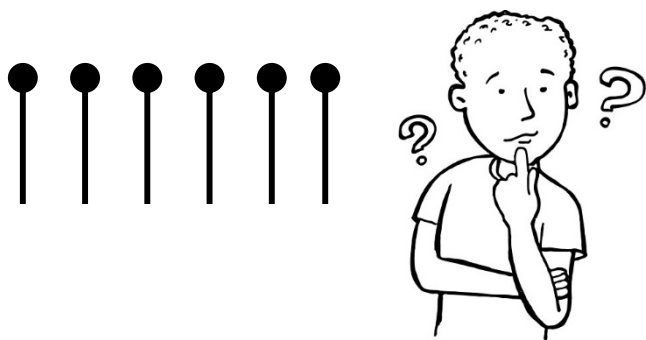


回溯问题的解空间

- **问题的解向量：**回溯法希望一个问题的解能够表示成一个 n 元向量 $(x_1, x_2, \dots, x_n)$ 的形式
 - **显约束：**对分量 x_i 的取值限定
 - **隐约束：**为满足问题的解而对不同分量之间施加的约束
 - **解空间：**对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间
- **注意：**同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小(存储量少，搜索方法简单)

回溯问题的解空间

- **例如：**有6根火柴，以之为边搭建4个等边三角形
 - 该问题易产生误导，它暗示是一个二维空间，为解决问题需拓展到三维。



- 对任意一个问题，解的表示方式和它相应的解隐含了解空间及其大小。

几个回溯法的例子



https://www.bilibili.com/video/BV1Bt411Q7yR/?spm_id_from=333.337.search-card.all.click&vd_source=d25fef25f767809aef481fc89452022

4后问题

问题： 在 4×4 的方格棋盘上放置4个皇后，使得没有两个皇后在同一行、同一列、也不在同一条45度的斜线上。问有多少种可能的布局？

解是4维向量： $\langle x_1, x_2, x_3, x_4 \rangle$

解： $\langle 2, 4, 1, 3 \rangle$

推广到8后问题

解： 8维向量，有92个。

例如： $\langle 1, 5, 8, 6, 3, 7, 2, 4 \rangle$ 是解

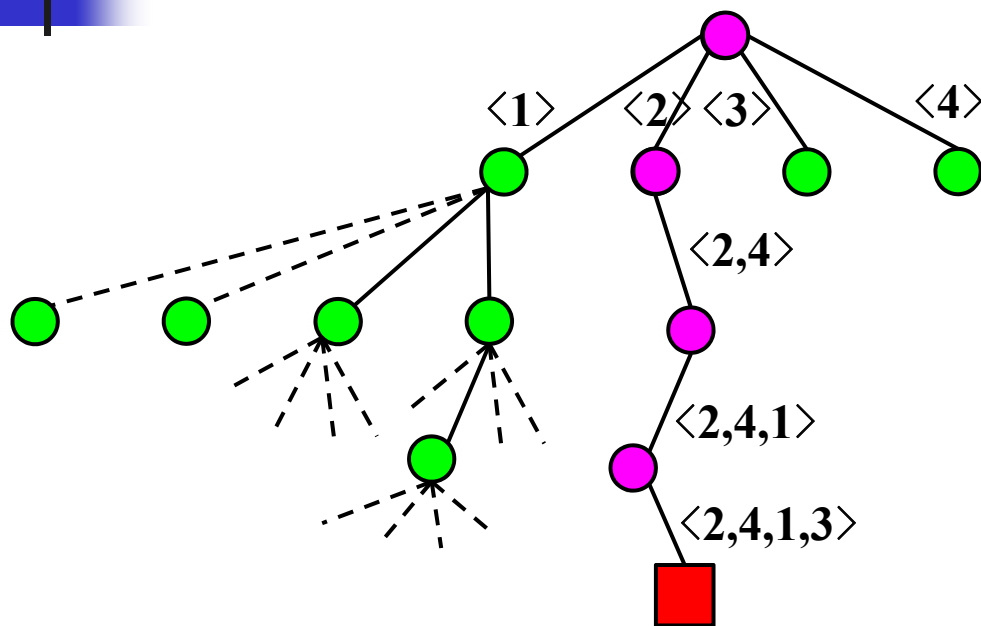
1	2	3	4	
	●			x_1
			●	x_2
●				x_3
		●		x_4



N后问题

N×N棋盘放置N皇后局面数	
N	局面总数
1	1
2	0
3	0
4	2
5	10
6	4
7	40
8	92
9	352
10	724
11	2680
12	14200
13	73712
14	365596

搜索空间：4叉树



第1层代表第1个皇后
可能选择的列号

- 每个结点有4个儿子，分别代表选择1, 2, 3, 4列位置
- 第*i*层选择解向量中第*i*个分量值
- 最深层的树叶是解
- 按深度优先次序遍历树，找到所有解

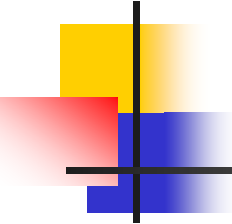
0-1背包问题

问题： 有 n 种物品，每种物品的重量和价值分别为 w_i , v_i 。如果背包的最大承重限制是 B ，每种物品至多放1个。怎么样选择放入背包的物品使得背包所装物品价值最大？



实例： $V=\{12,11,9,8\}$, $W=\{8,6,4,3\}$, $B=13$

最优解： $\langle 0,1,1,1 \rangle$, 价值：28, 重量：13



算法设计

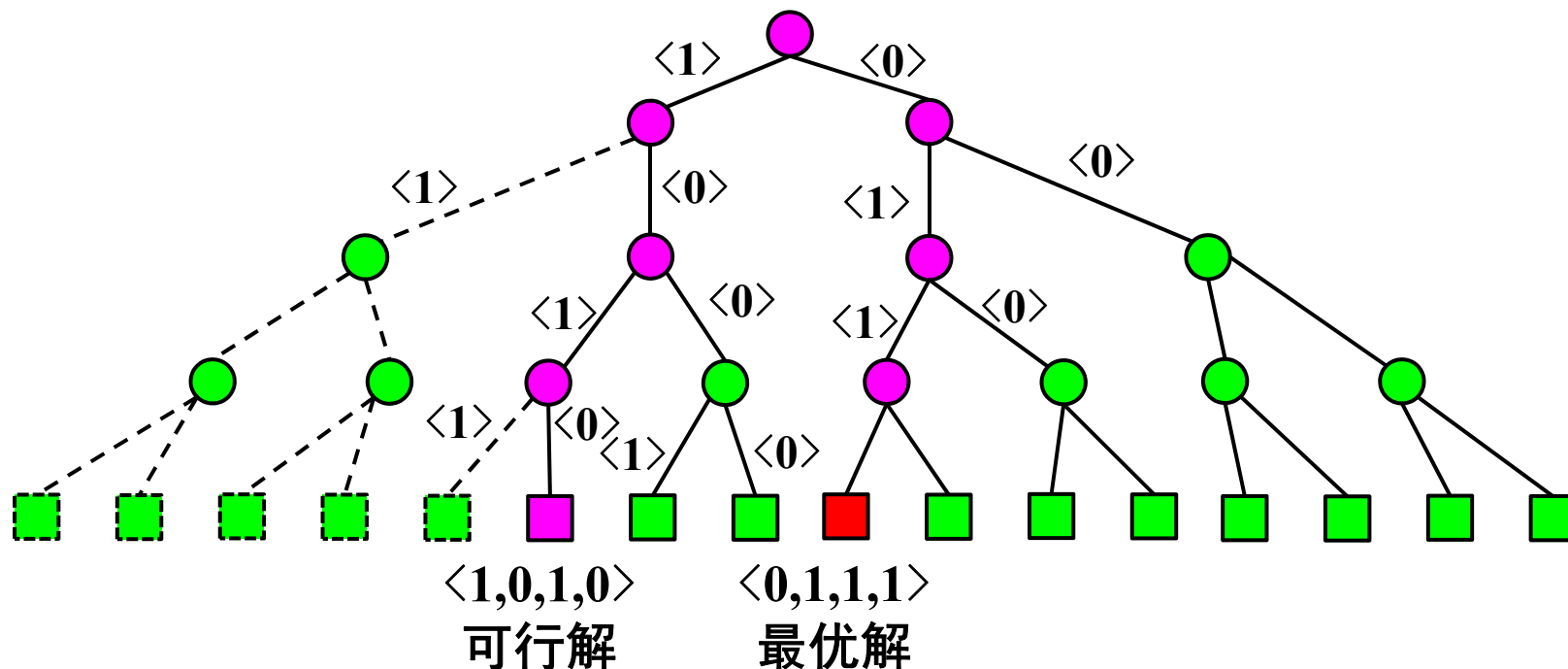
- **解：** n 维0-1向量 $\langle x_1, x_2, \dots, x_n \rangle$
 $x_i = 1 \Leftrightarrow$ 物品 i 选入背包
- **结点：** $\langle x_1, x_2, \dots, x_k \rangle$ （部分向量）
- **搜索空间：** 一棵0-1取值的二叉树，称为**子集树**，有 2^n 片树叶
- **可行解：** 满足约束条件(不超重)的解
- **最优解：** 可行解中价值达到最大的解

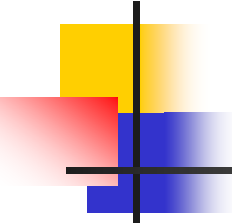
实例

- **输入：** $V=\{12,11,9,8\}$, $W=\{8,6,4,3\}$, $B=13$
- **2个可行解：**
 - $\langle 0,1,1,1 \rangle$, 价值：28, 重量：13
 - $\langle 1,0,1,0 \rangle$, 价值：21, 重量：12
- **最优解：** $\langle 0,1,1,1 \rangle$



- **实例：** $V=\{12,11,9,8\}$, $W=\{8,6,4,3\}$, $B=13$
- **搜索空间：** 子集树， 2^n 片树叶





旅行售货员问题

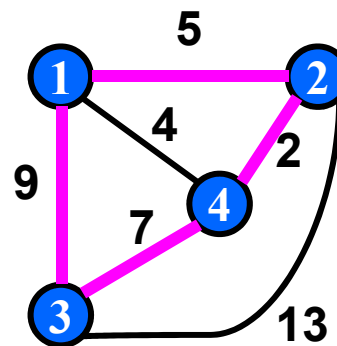
- **问题：** 一个售货员需要在 n 个城市销售商品，已知任两个城市之间的距离，求一条每个城市恰好经过一次的回路，使得总长度最小。
- **建模：** 城市集 $C=\{c_1, c_2, \dots, c_n\}$, 距离 $d(c_i, c_j)=d(c_j, c_i)$
- **求解：** $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$

实例

输入：

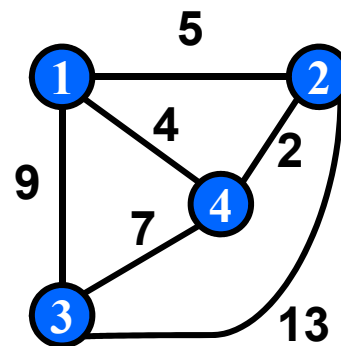
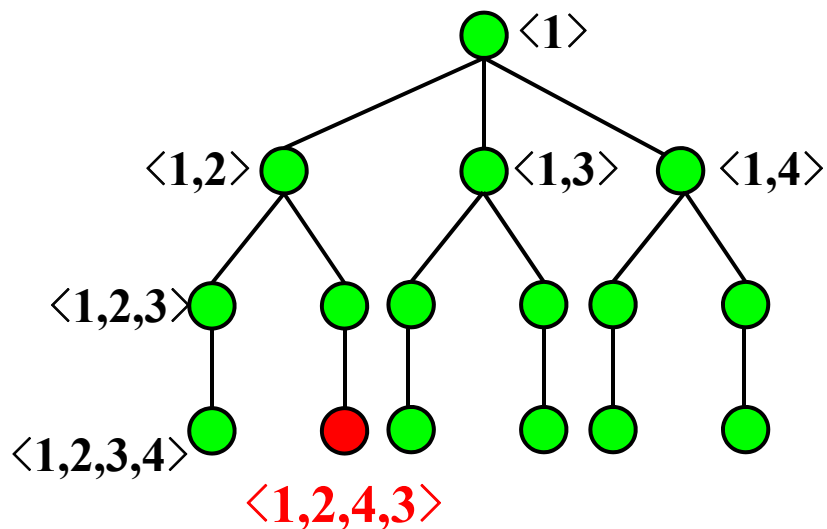
- $C=\{1,2,3,4\}$
- $d(1,2)=5, d(1,3)=9$
- $d(1,4)=4, d(2,3)=13$
- $d(2,4)=2, d(3,4)=7$



解： $\langle 1,2,4,3 \rangle$, 长度 $=5+2+7+9=23$

搜索空间

- 排列树，有 $(n-1)!$ 片树叶



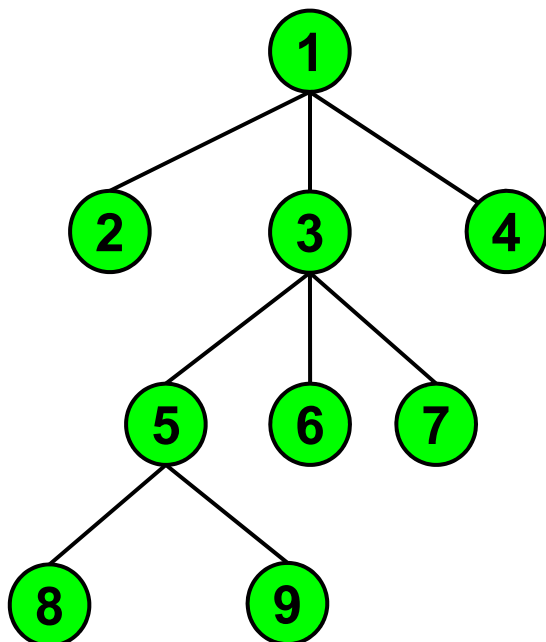
小结

问题	解性质	解向量	搜索空间	搜索方式	约束条件
n 后问题	可行解	$\langle x_1, x_2, \dots, x_n \rangle$ x_i :第 i 行列号	n 叉树	深度优先	彼此 不攻击
0-1背包 问题	最优解	$\langle x_1, x_2, \dots, x_n \rangle$ $x_i=0/1$ $x_i=1 \Leftrightarrow$ 选 i	子集树	深度优先	不超过 背包重量
TSP问题	最优解	$\langle k_1, k_2, \dots, k_n \rangle$ $1, 2, \dots, n$ 的排列	排列树	深度优先	选未经过 的城市
特点	搜索解	扩张 部分向量	树	跳跃式 遍历	约束条件 回溯判定

回溯法设计思想和适用条件

回顾一下

■ 深度与宽度优先搜索

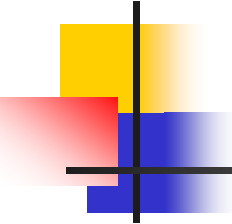


深度优先访问顺序:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow 7 \rightarrow 4$

宽度优先访问顺序:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$



回溯法基本设计思想

- **适用对象：**求解搜索问题和优化问题
- **搜索空间：**树，结点对应部分解向量，可行解在树叶上
- **搜索过程：**采用系统的方法隐含遍历搜索树
- **搜索策略：**深度/宽度优先、函数优先、宽深结合
- **结点分支判定条件：**
 - 满足约束条件—分支扩张解向量
 - 不满足约束条件—回溯到该结点的父结点
- **结点状态：**动态生成
 - 可以约定：白色结点（尚未访问）、灰色结点（正在访问）、黑色结点（该结点为根的子树遍历完成）
- **存储：**当前路径

结点状态例子

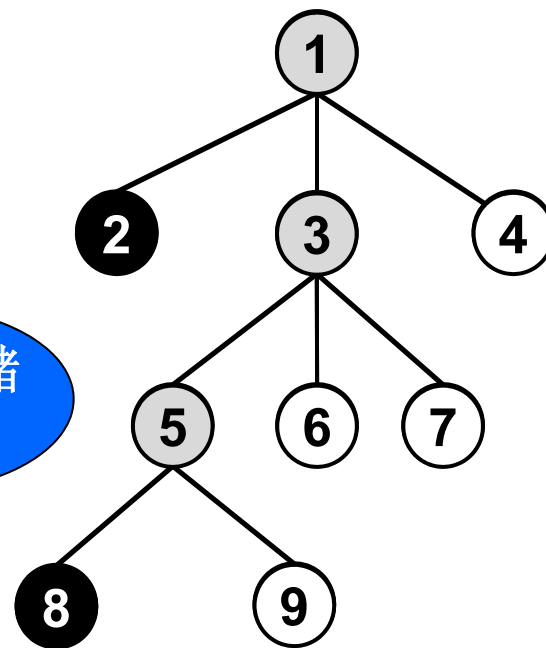
- 策略：深度优先

- 访问次序：

1→2→3→5→8

当前存储
的路径

- 已完成访问：2,8
- 已访问但未结束：1,3,5
- 尚未访问：9,6,7,4



回溯法适用条件

- 在结点 $\langle x_1, x_2, \dots, x_k \rangle$ 处

$P(x_1, x_2, \dots, x_k)$ 为真

$\Leftrightarrow$ 向量 $\langle x_1, x_2, \dots, x_k \rangle$ 满足某个性质（约束条件）

（例如： n 后中 k 个皇后放在彼此不攻击的位置）

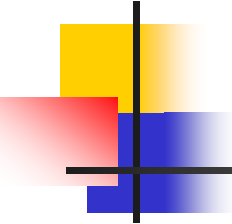
- 多米诺性质

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

- 逆否命题：

$$\neg P(x_1, x_2, \dots, x_k) \rightarrow \neg P(x_1, x_2, \dots, x_{k+1}) \quad 0 < k < n$$

作用： k 维向量不满足约束条件，扩张向量到 $k+1$ 维仍旧不满足，可以回溯！



一个实例

例：求不等式的整数解

$$5x_1 + 4x_2 - x_3 \leq 10, 1 \leq x_k \leq 3, k=1,2,3$$

$P(x_1, x_2, \dots, x_k)$ ：将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分，部分算术值小于等于10

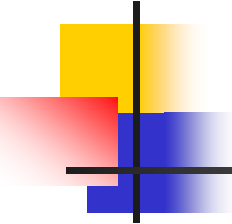
→不满足多米诺性质：

$$5x_1 + 4x_2 - x_3 \leq 10 \not\Rightarrow 5x_1 + 4x_2 \leq 10$$

通过变换使得问题满足多米诺性质：

令 $x_3 = 3 - x_3^*$ ，那么原不等式变换为

$$\begin{aligned} 5x_1 + 4x_2 + x_3^* &\leq 13 \\ 1 \leq x_1, x_2 \leq 3, 0 \leq x_3^* &\leq 2 \end{aligned}$$



归纳一下

- 回溯法适用条件：多米诺性质
- 回溯法设计步骤
 1. 定义解向量和每个分量的取值范围
 - 解向量为 $\langle x_1, x_2, \dots, x_n \rangle$
 - 确定 x_i 的取值集合为 X_i (显约束)
 2. 由 $\langle x_1, x_2, \dots, x_{k-1} \rangle$ 确定如何计算 x_k 取值集合 $S_k, S_k \subseteq X_k$
 3. 确定结点儿子的排列规则
 4. 判断是否满足多米诺性质
 5. 确定每个结点分支的约束条件
 6. 确定搜索策略：深度优先/宽度优先，...
 7. 确定存储搜索路径的数据结构

回溯法的实现及实例

回溯法两种实现

1. 递归实现

■ 算法ReBack(k)

1. if $k > n$ then $\langle x_1, x_2, \dots, x_n \rangle$ 是解
2. else while $S_k \neq \emptyset$ do
3. $x_k \leftarrow S_k$ 中最值
4. $S_k \leftarrow S_k - \{x_k\}$ // 取出来后, 肯定会导致取值集合变少
5. 计算 S_{k+1} // 再考虑下一个的取值集合
6. ReBack($k+1$)

■ 算法ReBacktrack(n)

■ 输入: n

■ 输出: 所有解

1. for $k \leftarrow 1$ to n 计算 X_k 且 $S_k \leftarrow X_k$
2. ReBack(1)

回溯法两种实现

2. 迭代实现

■ 算法Backtrack(n)

■ 输入: n

■ 输出: 所有解

1. 对于 $i=1,2,\dots,n$ 确定 X_i
2. $k \leftarrow 1$
3. 计算 S_k
4. **while** $S_k \neq \emptyset$ **do**
5. $x_k \leftarrow S_k$ 中最值; $S_k \leftarrow S_k - \{x_k\}$
6. **if** $k < n$ **then**
7. $k \leftarrow k+1$; 计算 S_k
8. **else** $\langle x_1, x_2, \dots, x_n \rangle$ 是解
9. **if** $k > 1$ **then** $k \leftarrow k-1$; **goto** 4

确定初
始取值

满足约束
分支搜索

回溯

装载问题

问题：有 n 个集装箱，需要装上两艘载重分别为 c_1 和 c_2 的轮船。 w_i 为第 i 个集装箱的重量，且 $w_1 + w_2 + \dots + w_n \leq c_1 + c_2$ 。问是否存在一种合理的装载方案把这 n 个集装箱装上船？如果有，给出一种方案。

